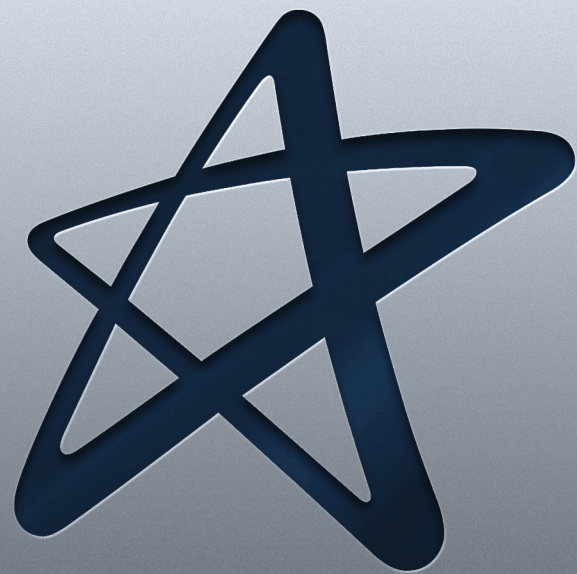


Compiladores e Interpretadores



Cruzeiro do Sul Virtual
Educação a distância

Material Teórico



Geração de Código para Comandos Condicionais

Responsável pelo Conteúdo:

Prof. Dr. Cleber Silva Ferreira da Luz

Revisão Textual:

Prof.^a Dr.^a Luciene Oliveira da Costa Granadeiro

UNIDADE

Geração de Código para Comandos Condicionais



- Introdução;
- Geração de Código para Referências a Estruturas de Dados;
- Geração de Código para Declarações de Controle e Expressões Lógicas;
- Geração de Código para Expressões Lógicas;
- Geração de Código para Chamadas de Procedimentos e Funções;
- Geração de Código para Matrizes.



OBJETIVO DE APRENDIZADO

- Apresentar os conceitos da geração de códigos com foco nos comandos de condicionais.



Orientações de estudo

Para que o conteúdo desta Disciplina seja bem aproveitado e haja maior aplicabilidade na sua formação acadêmica e atuação profissional, siga algumas recomendações básicas:



Assim:

- ✓ Organize seus estudos de maneira que passem a fazer parte da sua rotina. Por exemplo, você poderá determinar um dia e horário fixos como seu “momento do estudo”;
- ✓ Procure se alimentar e se hidratar quando for estudar; lembre-se de que uma alimentação saudável pode proporcionar melhor aproveitamento do estudo;
- ✓ No material de cada Unidade, há leituras indicadas e, entre elas, artigos científicos, livros, vídeos e *sites* para aprofundar os conhecimentos adquiridos ao longo da Unidade. Além disso, você também encontrará sugestões de conteúdo extra no item **Material Complementar**, que ampliarão sua interpretação e auxiliarão no pleno entendimento dos temas abordados;
- ✓ Após o contato com o conteúdo proposto, participe dos debates mediados em fóruns de discussão, pois irão auxiliar a verificar o quanto você absorveu de conhecimento, além de propiciar o contato com seus colegas e tutores, o que se apresenta como rico espaço de troca de ideias e de aprendizagem.

Introdução

Continuaremos o estudo sobre a fase de síntese do processo de compilação. Na unidade anterior, estudamos a geração de código intermediária, otimização e foi iniciado o estudo na geração de código predita. Nesta unidade, continuaremos o estudo da geração de código predita. Vale ressaltar que, para a geração de código intermediário, são usadas as técnicas de código de três endereços e a técnica P-Código. Após o uso dessas técnicas, é feita, então, a geração das instruções em nível de máquina. Assim, continuaremos a estudar a técnica de código de três endereços e a técnica P-Código. Começaremos o estudo pela geração de código para referenciar uma estrutura de dados.

Geração de Código para Referências a Estruturas de Dados

Na unidade anterior, vimos a geração de código a partir de código intermediário para expressões aritméticas simples e atribuição, que, inclusive, todos os valores eram constantes ou variáveis simples (variáveis de programas como x , ou temporários como $t1$). Neste momento, é necessário relembrarmos algumas informações importantes. Devemos lembrar como a técnica de três endereços trabalha. Por exemplo, vamos considerar a expressão $2*a+(b-3)$. Para essa expressão, temos a árvore sintática equivalente:

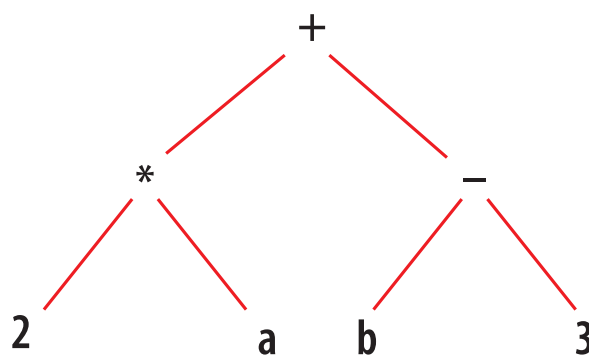


Figura 1 – Árvore sintática para $2*a+(b-3)$

Fonte: Adaptado de Louden (2004)

Para essa expressão, a técnica de código de três endereços realizaria os seguintes passos:

$$t1 = 2 * a$$

$$t2 = b - 3$$

$$t3 = t1 + t2$$

Devemos lembrar que a técnica de código de três endereços cria uma variável temporária que, para essa expressão, temos as variáveis **t1**, **t2** e **t3**. Na técnica de código de três endereços, é necessário que o compilador gere nomes para os temporários, que denominamos nesse exemplo como: **t1**, **t2** e **t3**.

Quando se gera uma instrução de máquina, devemos calcular o endereço de memória para a mesma. Nessa técnica, as variáveis simples foram identificadas apenas por nome – a tradução para o código-alvo exige que esses nomes sejam substituídos pelos endereços efetivos que poderiam ser registradores, endereços absolutos de memória (para globais) ou deslocamentos de registros de ativação. Esses endereços podem ser inseridos quando o código intermediário for gerado ou após a geração de código.

Devemos observar, também, que há muitas situações, entretanto, que exigem **cálculo de endereços** para localizar o endereço em questão, e esses cálculos devem ser expressões diretas, mesmo no código intermediário. Tais cálculos ocorrem na indexação de matrizes, nos campos de registros e nas referências de ponteiros. Vamos começar por código de três endereços para cálculos de endereços.

Para o código de três endereços, não existe tanta necessidade de novas operações – as operações aritméticas usuais, isto é, que frequentemente são usadas pode sem ser usadas para computar endereços, só que de formas a indicar os modos de endereçamento “endereço de” e “indireto”. Por exemplo, vamos supor que desejamos armazenar o valor 2 no endereço da variável *x* mais 10 bytes. Essa situação seria expressa em código de três endereços desta forma:

$$t1 = \&x + 10$$

$$*t1 = 2$$

Em P-código, é comum acrescentar novas instruções para expressar novos modos de endereçamento. Assim, para essa questão de cálculo de endereço, podemos adicionar as seguintes instruções:

- **ind**, assume que um endereço está no topo da pilha, adiciona o deslocamento ao endereço e substitui o endereço na pilha pelo valor no local resultante:

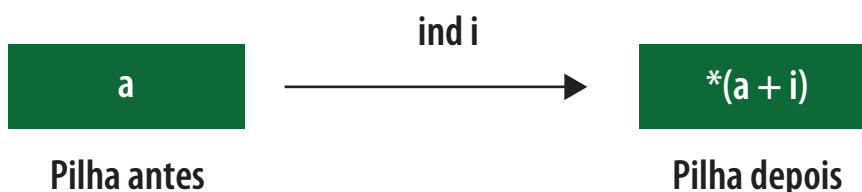


Figura 2 – Ação ind

Fonte: Adaptado de Louden (2004)

- **ixa** (endereço indexado), que recebe como parâmetro um fator de escala inteiro, assume um deslocamento no topo da pilha e um endereço de base logo abaixo, multiplica o deslocamento pelo fato da escala, adiciona o endereço de base. Logo:

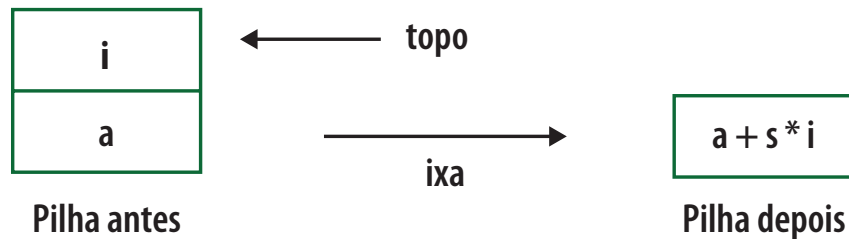


Figura 3 – Ação ixa
Fonte: Adaptado de Louden (2004)

Podemos dizer que essas duas instruções adicionadas e a instrução lda (que carrega o endereço) nos permitirão efetuar os mesmos cálculos e referência de endereços.

Geração de Código para Declarações de Controle e Expressões Lógicas

Agora, vamos estudar como é gerado o código para as declarações de controles e expressões lógicas. As principais declarações de controles que iremos estudar são *if* e *else* e *while*. Devemos observar que a geração de código intermediário para as declarações de controle também pode ser realizada pelo **código de três endereços** e **P-código**.

Vamos começar o estudo pela geração de código para as declarações *if* e *while*. Para isso, vamos considerar as duas formas de declarações *if* e *while* similares em diversas linguagens:

If-decl $\rightarrow$ **if** (exp) decl | **if** (exp) decl **else** decl

while-decl $\rightarrow$ **while** (exp) decl

Vale observar que as declarações foram feitas utilizando gramática livre de contexto. O maior problema para gerar as instruções de máquina para essas declarações é traduzir as características de controle estruturado em uma forma “sem estrutura” com saltos, que podem ser implementados diretamente. Estruturas de controle realizam saltos, isto é, se uma condição for satisfeita, determinado código é executado, caso contrário, outras instruções devem ser executadas (salto). Habitualmente, os compiladores geram código para tais declarações em uma forma padrão que permite o uso eficiente de um subconjunto dos saltos possíveis considerando uma arquitetura-alvo. A Figura 2 ilustra a organização de uma estrutura *if*. Já a Figura 3 ilustra a organização para uma declaração *while*. Em ambas as organizações, há apenas dois tipos de saltos – saltos incondicionais e saltos nos quais a condição é falsa; o caso verdadeiro é sempre um caso “seguir adiante” que não requer saltos. Isso indica que apenas duas instruções de saltos são exigidas no código intermediário.

Vamos estudar agora como seria o código de três endereços para as declarações de controle. Para isso, considere:

If (E) S1 else S2

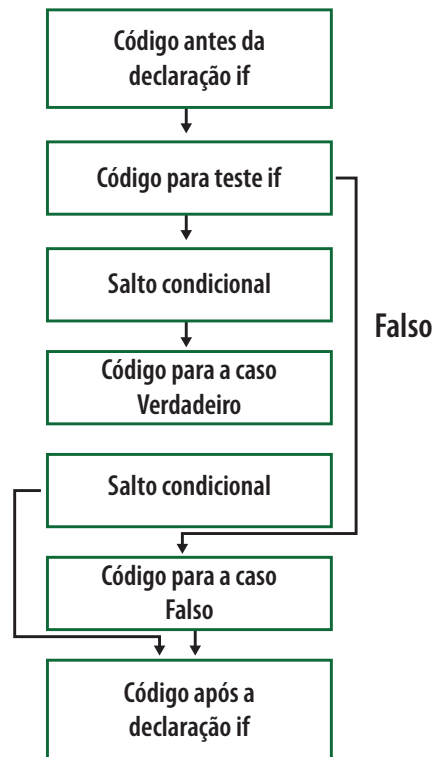


Figura 4 – Organização da estrutura *if*
Fonte: extraído de Louden (2004)

O padrão de código a seguir é gerado:

<código para avaliar E para t1>

If_false t1 goto L1

<código para S1>

goto L2

label L1

<código para S2>

label L2

Para uma declaração while:

while (E) S

Tal declaração produziria o padrão de código de três endereços a seguir:

label l1

<código para avaliar E para t1>

```
If_false t1 goto l2  
  
<código para S>  
  
goto l1  
  
label l2
```

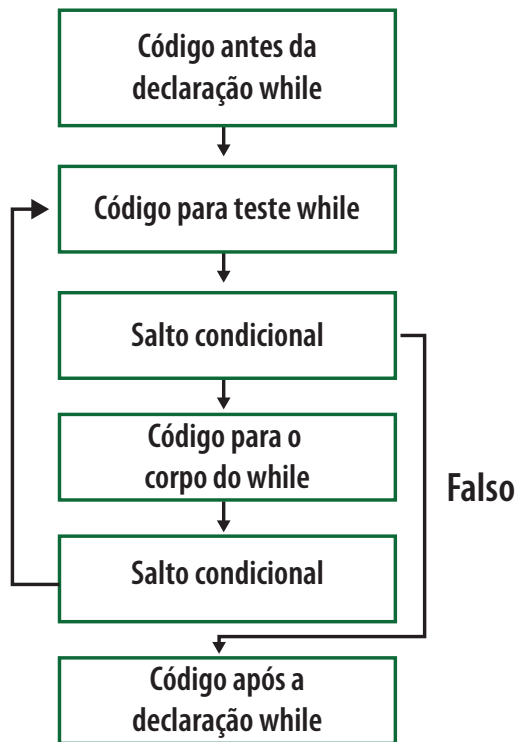


Figura 5 – Organização da estrutura *while*

Fonte: Adaptado de Louden (2004)

Na próxima seção, iremos estudar a geração de código para expressões lógicas.

Geração de Código para Expressões Lógicas

Nesta seção, iremos abordar a geração de código para expressões lógicas ou booleanas. Já foi dito por diversas vezes que, após a geração de código intermediário, são realizadas otimizações no código e, logo em seguida, é gerado o código-alvo. Agora, se o código intermediário tiver um tipo de dado booleano e operações lógicas como **and** e **or**, o valor de uma expressão booleana pode ser computado em código intermediário exatamente da mesma forma que uma expressão aritmética. Entretanto, a tradução em código-alvo geralmente requer que os valores booleanos sejam representados aritmeticamente, pois a maioria da arquitetura não apresenta um tipo de dados booleano interno. A forma padrão para isso é representar o valor booleano como falso 0 e verdadeiro 1. Nesse caso, os operadores de *bits* padrão **and** e **or** podem ser usados para computar os valores de uma expressão booleana na maioria das arquiteturas. Devemos observar que isso exige que o resultado das operações de comparação com `<` sejam normalizados zero ou um. Em algumas,

isso exige que zero ou 1 sejam carregados explicitamente, pois o operador de comparação apenas ajusta um código de condição. Assim, os saltos condicionais precisam ser gerados para carregar o valor apropriado.

Podemos utilizar a ideia de **curto-circuito**. Uma operação lógica é um curto-circuito se não puder avaliar o seu segundo argumento. Vamos exemplificar essa situação, considere **a** como uma expressão booleana computada como falsa, a expressão booleana **a and b** pode ser determinada de imediato como falsa sem avaliar **b**. Similarmente, se **a** for verdadeiro, então **a or b** pode ser determinada como verdadeira sem avaliar **b**. As operações de curto-circuito são extremamente úteis para o codificador, pois a avaliação da segunda expressão provocaria um erro se um operador não fosse de curto-circuito (LOUDEN, 2004). Considere a instrução a seguir:

if ((p != NULL) && (p → val == 0)) ...

Se **p** for nulo, será provocado um erro de memória.

Operadores booleanos de curto-circuito são similares à declaração *if*, exceto que retornam valores e frequentemente são definidos com base em expressões *if* como:

a and b é próximo a **a if a then b else false**

Similarmente,

a or b é próximo a **a if a then true else b**

Agora, para gerar o código que garante a avaliação da segunda subexpressão quando necessário, precisamos usar saltos exatamente da mesma maneira que no código para as declarações *if*. Por exemplo, o P-código de curto-circuito para a expressão em C **(x != 0) && (y == x)** é:

```
fpj L1
lod x
ldc 0
neq
lod y
equ
ujp L2
lab L1
lod false
lab L2
```

Na próxima seção, é abordada a geração de código para chamadas de procedimentos e funções.

Geração de Código para Chamadas de Procedimentos e Funções

Mais uma vez, vamos descrever a forma de código intermediário para posteriormente gerar o código-alvo. Para descrever o código intermediário para procedimentos de funções, precisamos descrever dois mecanismos: a **definição** de uma função / procedimento e a **ativação** da função / procedimento. Uma **definição** cria um nome de função, parâmetros e código, mas a função não é executada naquele ponto. Uma **ativação** cria os valores para os parâmetros e efetua um salto para o código da função, que é executada e retorna um valor. Devemos observar que o ambiente onde ocorrerá a execução não é conhecido quando o código da função é criado.

As instruções (LOUDEN, 2004): o código intermediário para uma definição precisa conter uma instrução que marque o início, ou **ponto de entrada** do código para a função, e uma instrução que marque o final ou ponto de retorno da função. Podemos realizar um esboço da seguinte forma:

Instrução de entrada

<código para o corpo da função >

Instrução de retorno

Similarmente, uma ativação de função precisa de uma instrução para indicar o início da computação dos argumentos e de uma instrução de ativação que indique o ponto em que os argumentos foram construídos e para onde pode ser efetuado o salto para o código da função:

Instrução de argumento de início da computação

<código para computar os argumentos>

Instrução de ativação

Para a geração de código, também possível utilizar a técnica de três endereços e o P-Códigos. Vamos analisar a geração de código utilizando a técnica de três endereços. Na técnica de código de três endereços, a instrução de entrada precisa dar um nome para o ponto de entrada do procedimento, semelhante à instrução *label*, assim, ela é uma instrução de um endereço, que denominamos simplesmente de **entry**. Semelhantemente, denominaremos a instrução de retorno **return**. Essa instrução também é de um endereço: ela precisa dar o nome do valor de retorno. Vamos considerar a seguinte instrução:

```
Int g (int a, int b)
{
    return a + b + 1;
}
```

Essa instrução seria traduzida na técnica de três endereços da seguinte forma:

```
entry g  
t1 = a + b  
t2 = t1 + 1  
return t2
```

Vale ressaltar que *entry* é utilizada para nomear ou renomear uma função.

Para uma ativação, precisamos de três instruções distintas de três endereços: uma para indicar início da computação de argumento, que iremos denominar *begin_args*; uma instrução usada repetidamente para especificar os nomes dos valores de argumentos, que iremos denominar de *arg*; e finalmente a instrução de ativação, que iremos denominar de *call* (LOUDEN, 2004).

Considerando a função *g* para a seguinte situação:

$g(2 + 3, 4)$

é traduzido para o código de três endereços:

```
begin_args  
t1 = 2 + 3  
arg t1  
arg 4  
call g
```

Agora, revemos como a função *g* ficaria em P-código. A instrução de entrada em P-código é *ent*, e a instrução de retorno *ret*; dessa forma:

```
ent f  
lod x  
lod y  
adi  
ldc 1  
adi  
ret
```

Devemos observar que a instrução *ret* não requer parâmetro para indicar o valor de retorno; assume-se que o valor de retorno fica no topo da pilha da máquina P-máquina no retorno.

As instruções de P-código para uma ativação são a instrução *mst* e a instrução *cup*. A instrução *mst* – “marca pilha” – corresponde à instrução de três endereços

`begin_args`. O motivo do nome “marca pilha” é o que o código-alvo gerado por essa instrução ajusta o registro de ativação para a nova ativação da pilha. A instrução de P-código *cup* é a instrução de “ativar procedimento de usuário”, e corresponde diretamente à instrução *call* do código de três endereços. Considerando a função, para o P-Código temos:

```
mst
ldc 2
ldc 3
adi
ldc 4
cup f
```

Geração de Código para Matrizes

A geração de código para matrizes requer a indexação de uma variável de matriz por uma expressão para obter uma referência ou valor de um elemento único da matriz, como no código em C:

```
int a[tamanho];
int i
int j
.
.
.
a[i + 1] = a[ j * 2 ] + 3;
```

Nessa atribuição, a indexação de *a* pela expressão $i + 1$ produz um endereço, e a indexação de *a* pela expressão $j * 2$ produz o valor no endereço computado pelo tipo de elemento de *a*. Como as matrizes são armazenadas sequencialmente na memória, cada endereço deve ser computado a partir do endereço de base de um deslocamento que dependa linearmente do valor do índice.

Vamos, agora, estudar o cálculo de endereço para matriz no formato de código de três endereços. Para isso, vamos assumir que o “endereço” de uma variável de matriz é o seu endereço base. Dessa forma, se *a* for uma matriz, `&a` no código de três endereços será o mesmo que `endereço_base(a)`, e em P-código.

```
lda a
```

carrega o endereço de base *a* na pilha da P-código. Como um cálculo de referência de matriz, depende do tamanho do tipo de dados do elemento na máquina-alvo (LOUDEN, 2004).

Uma forma possível de expressar a referência de uma matriz no código de três endereços é introduzir novas operações, uma que captura o valor de um elemento de matriz:

```
Int t2
```

```
t2 = a[t1]
```

E uma que atribui um valor ao endereço de um elemento de matriz:

```
a[t2] = t1
```

Com essa terminologia, não é necessário expressar a computação do endereço. Por exemplo, a declaração em código-fonte:

```
int a[tamanho]
```

```
int i
```

```
int j
```

```
a[i + 1] = a[ j * 2] + 3
```

seria traduzida nas instruções de três endereços como:

```
t1 = j * 2
```

```
t2 = a[t1]
```

```
t3 = t2 + 3
```

```
t4 = i + 1
```

```
a[t4] = t3
```

Podemos, também, utilizar outro modo, por exemplo, observe o código:

```
int a [tamanho]
```

```
t2 = a[t1]
```

Podemos, também, escrever:

```
t3 = t1 * elemen_size(a)
```

```
t4 = &a + t3
```

```
t2 = * t4
```

e a atribuição:

```
a[t2] = t1
```

pode ser escrita como:

```
t3 = t2 * eleme_size(a)
```

```
t4 = &a + t3
```

```
*t4 = 1
```

Finalmente, como um exemplo mais complexo, a declaração em código-fonte:

```
int a[tamanho]
```

```
int i
```

```
int j
```

```
a[ i + 1] = a[ j * 2 ] + 3
```

é traduzida para as instruções de três endereços:

```
t1 = j * 3
```

```
t2 = t1 * elem_size (a)
```

```
t3 = &a + t2
```

```
t4 = *t3
```

```
t5 = t4 + 3
```

```
t6 = i + 1
```

```
t7 = t6 * elem_size(a)
```

```
t8 = &a + t7
```

```
*t8 = t5
```

Agora, vamos estudar como é a geração de código para matrizes em P-Código.

Em P-código, podemos utilizar as instruções *ixa* e *ind* adicionadas anteriormente.

Vamos considerar a instrução:

```
t2 = a[t1]
```

é escrita em P-Código como:

```
lda t2
```

```
lda a
```

```
lod t1
```

```
ixa elem_size (a)
```

```
ind 0
```

```
sto
```

e a atribuição de matriz:

```
a[t2] = t1
```

é escrita em P-Código como:

```
lda a
lod t2
ixa elem_size(a)
lod t1
sto
```

Finalmente, o exemplo anterior mais complexo:

```
a[i + 1] = a[j + 2] + 3;
```

é traduzido para as instruções em P-Código que seguem:

```
lda a
lod i
ldc 1
adi
ixa elem_size(a)
lda a
lod j
ldc 2
mpi
ixa_elem_size(a)
ind 0
ldc 3
adi
sto
```

Vale lembrar que o P-Código requer muito mais instruções que o P-Código do que a técnica de código de três endereços.



Em Síntese

O processo de compilação é composto por duas fases, a fase de análise e a de síntese. Nesta unidade, é estudada a geração de código, a fase de síntese. A fase de síntese é composta pelas etapas de geração de código intermediário, otimização e geração de código predita. O foco desta unidade é estudar a geração de código predita. Nesta unidade, são descritos mecanismos para a geração de código para estruturas de controle, chamada de função, expressões booleanas e matrizes.

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Livros

Construindo Compiladores

COOPER, K. D.; TORCZON, L. *Construindo Compiladores*. 2. ed. Rio de Janeiro: Elsevier, 2014.

Optimizing Compilers for Modern Architectures: a Dependence-based Approach

KENNEDY, K.; ALLEN, J. R. *Optimizing compilers for modern architectures: a dependence-based approach*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2002.

Leitura

Basics of Compiler Design

MOGENSEN, T. Æ. *Basics of Compiler Design*. 2010.

<http://bit.ly/2XMHZCu>

Compiler Construction

WAITE, W. M.; GOOS, G. *Compiler Construction*. 1996.

<http://bit.ly/2QQi80N>

Referências

AHO, A. V. **Compiladores: Princípios, Técnicas e Ferramentas**. 2. ed. Pearson, 2007. (*e-book*)

APPEL, A. W. **Modern Compiler Implementation In Java**. 2. ed. New York: Cambridge University Press, 2002.

BERGMANN, S. D. **Compiler Design: Theory, Tools, and Examples**, 2016. Disponível em: <<http://www.cs.cmu.edu/~aplatzer/course/Compilers/waitegoos.pdf>>. (*e-book*)

LOUDEN, K. C. **Compiladores: Princípios e Práticas**. São Paulo: Pioneira Thomson Learning, 2004.

PRICE, A. M. A. **Implementação de Linguagens de Programação: Compiladores**. 2. ed. Porto Alegre: Sagra Luzzatto, 2001.



Cruzeiro do Sul
Educatonal